



Lab_4_VibeCodin...
PDF

https://github.com/SETAPESU26/16_hangman

Yep bro, I checked both the **Lab 4 handout** and your assigned GitHub repo.

Your assignment is **Scenario 16 — Hangman Challenge**. The handout says you need to inspect the existing code, reproduce the bug, use Vibe Coding/LLM prompts to fix it and add the required features, then test and submit the before/after videos plus the complete LLM chat history.  Lab_4_VibeCoding_Student_handou...

Your repo contains:

- `main.py`
- `game.py`
- `words.py`
- `stats.py`
- `requirements.txt`
- `README.md`  GitHub

What you have to implement

There are **4 tasks** in the README:

1. Fix repeated guesses

- Repeated wrong letter should NOT reduce another life.
- Repeated correct letter should NOT count as another guess.
- Every distinct letter should affect the round only once.  GitHub

2. Session statistics

- Track rounds played.
- Track rounds won.
- Track best streak.
- Starting a new round should reset only round-specific data.  GitHub

3. Difficulty + scoring

- Add difficulty choices.
- Difficulty changes available lives and scoring.
- Category selection must remain.

- Using a hint must consistently affect the score.  GitHub

4. Input handling

- Handle invalid letters.
- Handle repeated commands.
- Handle category selection.
- Handle hints.
- Invalid input must not modify game state.
- Feedback shouldn't be duplicated.  GitHub

The README also specifically says **don't replace the project with an unrelated implementation**, don't add persistent storage such as CSV/JSON/SQLite, and keep the code modular.  GitHub

Important

Before changing anything, your lab requires you to **run the original program and record the 10-second "before" video showing the broken behavior**. The README specifically says to reproduce the Task 1 issue before modifying the code.  GitHub

If you want, I can take you through this **exactly step-by-step from zero: Clone → open in VS Code → run original → reproduce bug → record before video → give me the code → I'll give you the exact Vibe Coding prompts one by one → test every task → Git commit/push → record after video → prepare submission.**

okay now guide me step by step what I should do

Absolutely bro. We'll do it **step by step**, and I'll tell you exactly what to click/type at each stage. Don't change the code randomly yet.

Your Lab 4 instructions require: clone/fork the assigned repo, run the original Python game, record a **10-second before video**, fix the broken code and add the README features using prompts, record the **10-second after video**, push your updated code, and submit the videos, code, and exported chat history.  Lab_4_VibeCoding_Student_handou...

PART 1 — Get the project onto your laptop

Your assigned repository is:

SETAPESU26/16_hangman

Step 1 — Open the GitHub repository

Open:

[Your Hangman GitHub repository ↗](#)

You should see files something like:

⟨/⟩ Plain text

```
main.py
game.py
words.py
stats.py
requirements.txt
README.md
```

Do NOT edit anything on GitHub directly.

PART 2 — Fork the repository

Your lab says to clone or fork the assigned repo.  [Lab_4_VibeCoding_Student_handou...](#)

Since the original repo belongs to SETAPESU26 , the safest approach is **Fork**.

Step 2 — Click Fork

On the GitHub page:

Fork → Create new fork

You should get something like:

⟨/⟩ Plain text

```
Owner: YOUR_GITHUB_USERNAME
Repository name: 16_hangman
```

Keep the repository name as:

```
</> Plain text
```

```
16_hangman
```

Then click:

Create fork

Now you should have:

```
</> Plain text
```

```
YOUR_USERNAME/16_hangman
```

This is the repository you will modify.

 Remember: **Do NOT create a Pull Request to SETAPESU26** . Your handout explicitly says not to raise a PR to the main repo.  [Lab_4_VibeCoding_Student_handou...](#)

PART 3 — Clone your fork

Now open **Terminal** on your Mac.

You can use:

Command + Space → type Terminal → Enter

Go somewhere convenient, for example Desktop:

```
</> Bash
```

```
cd ~/Desktop
```

Then clone **your fork**, not the original repository.

It will look like:

</> Bash

```
git clone https://github.com/YOUR_USERNAME/16_hangman.git
```

Replace `YOUR_USERNAME` with your actual GitHub username.

For example:

</> Bash

```
git clone https://github.com/yourusername/16_hangman.git
```

Then:

</> Bash

```
cd 16_hangman
```

Check the files:

</> Bash

```
ls
```

You should see:

</> Plain text

```
main.py
game.py
words.py
stats.py
requirements.txt
README.md
```

PART 4 — Open the project in VS Code

From inside the project folder, run:

```
</> Bash
```

```
code .
```

If `code .` doesn't work, open **VS Code manually**:

VS Code → File → Open Folder → Desktop → 16_hangman

You should now see the project on the left side.

PART 5 — Read README FIRST

This is important because your lab specifically tells you to go through the README to understand the deliverables.  [Lab_4_VibeCoding_Student_handou...](#)

In VS Code, click:

```
</> Plain text
```

```
README.md
```

Read it once.

The project has **4 tasks**:

Task 1

Fix repeated guesses.

Task 2

Add session statistics.

Task 3

Add difficulty and scoring.

Task 4

Improve input handling.

Don't start coding yet.

PART 6 — Install requirements

Open the VS Code terminal:

Terminal → New Terminal

Make sure you're inside:

</> Plain text

16_hangman

Run:

</> Bash

python3 --version

You should get something like:

</> Plain text

Python 3.x.x

Then install the requirements:

</> Bash

pip3 install -r requirements.txt

If that doesn't work, try:

</> Bash

```
python3 -m pip install -r requirements.txt
```

PART 7 — Run the ORIGINAL program

This is very important.

Do not modify any code before this.

Run:

</> Bash

```
python3 main.py
```

If that doesn't work, try:

</> Bash

```
python main.py
```

The lab specifically requires you to run the Python-based game before making changes.



Lab_4_VibeCoding_Student_handou...

PART 8 — Reproduce the existing bug

According to the assignment, Task 1 is the broken behavior you need to demonstrate.

You need to test **repeated guesses**.

For example, if the game asks you to guess a letter:

</> Plain text

```
Guess a letter:
```

Enter:

</> Plain text

a

Then enter:

</> Plain text

a

again.

Watch what happens.

Try both situations:

Case A — repeated WRONG letter

For example:

</> Plain text

Guess: z

Guess: z

Check whether your lives decrease twice.

Case B — repeated CORRECT letter

For example:

</> Plain text

Guess: e

Guess: e

Check whether the game processes it again.

The expected fixed behavior is:

A repeated letter should not affect the round a second time.

PART 9 — NOW RECORD YOUR BEFORE VIDEO

Your handout specifically says:

Record 10 seconds of video before making any changes.  Lab_4_VibeCoding_Student_handou...

So now record approximately **10 seconds** showing:

1. Terminal/VS Code.
2. The Hangman program running.
3. You entering the repeated guess.
4. The incorrect behavior happening.

Important

Don't record a huge video.

Just approximately:

 Plain text

```
Game running
  ↓
Make first guess
  ↓
Repeat same guess
  ↓
Show bug
```

Save it as:

 Plain text

```
before.mp4
```

or

</> Plain text

Hangman_Before.mp4

Keep this file safe.

PART 10 — NOW WE START VIBE CODING

This is where your assignment becomes interesting.

The handout says:

Write prompts and fix the broken code.

And then:

Write prompts to add all the features listed in the README one by one.

 Lab_4_VibeCoding_Student_handou...

Do NOT ask the AI to rewrite the entire project in one prompt.

We'll make separate prompts for:

</> Plain text

Prompt 1 → Task 1

Prompt 2 → Task 2

Prompt 3 → Task 3

Prompt 4 → Task 4

This also gives you a clean chat history for your submission.

VERY IMPORTANT — Keep the Chat History

Create one ChatGPT conversation for this assignment.

Use it for all four tasks.

Your final submission requires:

</> Plain text

Chat history exported as doc/pdf

The handout explicitly lists that as a deliverable.  Lab_4_VibeCoding_Student_handou...

So don't delete the conversation.

PART 11 — Before we give the first coding prompt

We need to give the AI enough context.

Your first prompt should tell the AI:

- this is an existing Hangman project
- don't rewrite unnecessarily
- inspect the existing code
- identify the repeated-guess bug
- fix only Task 1
- preserve existing functionality
- don't implement Tasks 2–4 yet

Your first Vibe Coding prompt will be:

</> Plain text

I am working on an existing Python Hangman project for my lab assignment.

Please first inspect the existing project structure and understand how the current game works.

For now, ONLY fix Task 1 from the README: repeated guesses.

Requirements:

- A repeated wrong letter must not reduce the player's lives again.
- A repeated correct letter must not be counted as a new guess.

- Every distinct guessed letter should affect the current round only once.
- Preserve the existing game behavior and structure.
- Do not implement the other tasks yet.
- Do not rewrite the project unnecessarily.
- Make the smallest clean changes required.
- After making the changes, explain which files you modified and why.

But don't send this yet if you haven't opened the project in your AI coding environment.



Your workflow from here

We'll follow this exact order:

</> Plain text

```
STEP 1  → Fork repo
      ↓
STEP 2  → Clone repo
      ↓
STEP 3  → Open in VS Code
      ↓
STEP 4  → Read README
      ↓
STEP 5  → Install requirements
      ↓
STEP 6  → Run original game
      ↓
STEP 7  → Reproduce bug
      ↓
STEP 8  → Record BEFORE video
      ↓
STEP 9  → Vibe Coding Prompt #1
      ↓
STEP 10 → Test Task 1
      ↓
```

For now, stop after the BEFORE video. Don't modify the code yet.

Your immediate task

Do these 5 things only:

</> Plain text

1. Fork 16_hangman
2. Clone your fork
3. Open it in VS Code
4. Run python3 main.py
5. Reproduce and record the repeated-guess bug

Then we'll move to the actual Vibe Coding part.

Where are you right now: Forked , Cloned , or VS Code open ?

1 selection

okay now I am done with before video...now I want uh to give
vibe coding and separate prompts for each task

Yes bro 👍 Since your **before video is done**, now we should do the Vibe Coding properly with **4 separate prompts**, one task at a time. This is also aligned with the lab instruction to add the README features **one by one**.  Lab_4_VibeCoding_Student_handou...

Important: Don't give all 4 tasks in one prompt. We want the chat history to clearly show:

Prompt 1 → Fix Task 1 → Test → Prompt 2 → Test → Prompt 3 → Test → Prompt 4 → Test

🟢 PROMPT 1 — Task 1: Repeated Guesses

Give your Vibe Coding tool this **exact prompt**:

Vibe Coding Prompt 1 — Repeated Guesses



I am working on this existing Python Hangman project. Please inspect the existing code and README before making changes.

For this prompt, work **ONLY** on Task 1 from the README: Fix repeated guesses.

Requirements:

- A repeated wrong letter must **NOT** reduce the player's lives again.

- A repeated correct letter must NOT be counted as another guess.
- Every distinct guessed letter should affect the current round only once.
- The existing game flow and functionality should be preserved.
- Do not implement Task 2, Task 3, or Task 4 yet.
- Do not rewrite the project unnecessarily.
- Make the smallest clean and modular changes needed.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- After making the changes, explain:
 1. Which files you changed.
 2. What was causing the repeated-guess problem.
 3. How your fix prevents repeated guesses from affecting the round twice.
 4. How I can test the fix manually.

After it finishes

Don't immediately move to Task 2.

Run the game and test:

Test 1 — Wrong repeated letter

</> Plain text

Guess: z

Guess: z

Check:

</> Plain text

First z → affects game

Second z → should NOT reduce life

Test 2 — Correct repeated letter

</> Plain text

Guess: a

Guess: a

Check that the second a doesn't count again.

If Task 1 works, continue.

PROMPT 2 — Task 2: Session Statistics

Now give it this **separate prompt**:

Vibe Coding Prompt 2 — Session Statistics



Now work ONLY on Task 2 from the README: Add session statistics.

Please first inspect the current code after the Task 1 changes and understand the existing game flow.

Implement the following:

- Track the number of rounds played during the current session.
- Track the number of rounds won during the current session.
- Track the player's best winning streak during the current session.
- Starting a new round must reset only round-specific data.
- Session statistics must NOT be reset when starting a new round.
- Update the statistics at the appropriate points when a round is won or lost.
- Display the session statistics in an appropriate place in the existing game flow.
- Preserve all functionality from Task 1, including the repeated-guess fix.
- Do not implement Task 3 or Task 4 yet.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- Keep the existing project structure and make clean, modular changes rather than rewriting the project.

After making the changes, explain:

1. Which files you changed.
2. How rounds played, rounds won, and best streak are tracked.
3. How you made sure starting a new round does not erase session statistics.

4. How I can manually test all three statistics.

Test Task 2

Play multiple rounds.

For example:

</> Plain text

Round 1 → WIN

Round 2 → LOSE

Round 3 → WIN

Round 4 → WIN

You should be able to verify that the statistics change appropriately.

Also test that starting a new round **doesn't reset the session statistics**.

PROMPT 3 — Task 3: Difficulty + Scoring

Now move to Task 3.

Give it this:

Vibe Coding Prompt 3 — Difficulty and Scoring



Now work **ONLY** on Task 3 from the README: Add difficulty selection and scoring.

Please inspect the current implementation and preserve all functionality already implemented in Tasks 1 and 2.

Implement the requirements from Task 3:

- Add difficulty choices to the game.
- Difficulty must affect the number of available lives.
- Difficulty must affect scoring.
- Keep the existing category selection functionality.

- Using a hint must consistently affect the player's score as specified by the README.
- Make sure the scoring behavior is consistent and predictable.
- Difficulty selection should integrate cleanly with the existing game flow.
- Preserve the repeated-guess protection from Task 1.
- Preserve the session statistics from Task 2.
- Do not implement Task 4 yet.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- Do not replace the existing Hangman implementation with an unrelated implementation.
- Make the smallest clean and modular changes necessary.

After making the changes, explain:

1. Which files you changed.
2. What difficulty options were added.
3. How each difficulty changes lives.
4. How each difficulty changes scoring.
5. How hints affect the score.
6. How I can manually test difficulty, category selection, scoring, and hints.

Test Task 3

Test each difficulty.

Check:

</> Plain text

Difficulty selection

↓

Different number of lives

↓

Different scoring

Then test categories.

Then use a hint and check that the score changes consistently.

PROMPT 4 — Task 4: Input Handling

Finally, give it Task 4.

Vibe Coding Prompt 4 — Input Handling



Now work ONLY on Task 4 from the README: Improve input handling.

Please inspect the current implementation and preserve all functionality from Tasks 1, 2, and 3.

Implement all input-handling requirements from Task 4:

- Handle invalid letter input safely.
- Handle repeated commands safely.
- Handle category selection correctly.
- Handle hint input correctly.
- Invalid input must NOT modify the game state.
- Repeated input should not accidentally change lives, score, guessed letters, streaks, or other game state.
- Feedback/messages should not be duplicated.
- Make sure valid inputs continue to work normally.
- Preserve the repeated-guess fix from Task 1.
- Preserve session statistics from Task 2.
- Preserve difficulty and scoring from Task 3.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- Do not rewrite the project unnecessarily.
- Keep the implementation clean, modular, and consistent with the existing project.

After making the changes, explain:

1. Which files you changed.
2. What invalid inputs are now handled.
3. How repeated commands are handled.
4. How invalid input is prevented from changing game state.
5. How duplicate feedback was prevented.
6. Give me a complete manual testing checklist for Task 4.

🔥 VERY IMPORTANT — Don't just trust the AI

After each prompt, you need to test the game yourself.

Your workflow should look like this:

⟨⟩ Plain text

```
TASK 1
  ↓
Give Prompt 1
  ↓
AI changes code
  ↓
Run the game
  ↓
Test repeated guesses
  ↓
✅ Working?
  ↓
TASK 2
  ↓
Give Prompt 2
  ↓
Test statistics
  ↓
TASK 3
  ↓
```

This is much better for your lab because the assignment specifically asks you to use prompts to fix the broken code and then add the README features **one by one**.

 Lab_4_VibeCoding_Student_handou...

⚠️ One more thing

Don't modify the prompts after giving them unless the Vibe Coding tool gives an error or misunderstands the existing code.

If it responds with something like:

"I've implemented Task 1. Here are the changes..."

send me that response, and I'll tell you **exactly what to test next** before we move to Prompt 2.

So right now:

👉 Give **Prompt 1 only** to your Vibe Coding tool.

Then paste its response here (or send a screenshot), and we'll verify **Task 1** before touching Task 2.

I am working on this existing Python Hangman project. Please inspect the existing code and README before making changes.

For this prompt, work **ONLY** on Task 1 from the README: Fix repeated guesses.

Requirements:

- A repeated wrong letter must NOT reduce the player's lives again.
- A repeated correct letter must NOT be counted as another guess.
- Every distinct guessed letter should affect the current round only once.
- The existing game flow and functionality should be preserved.
- Do not implement Task 2, Task 3, or Task 4 yet.
- Do not rewrite the project unnecessarily.
- Make the smallest clean and modular changes needed.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- After making the changes, explain:
 1. Which files you changed.
 2. What was causing the repeated-guess problem.
 3. How your fix prevents repeated guesses from affecting the round twice.
 4. How I can test the fix manually.

Yes bro — **that is the correct Prompt 1.** 🧑🏻‍🔧

Now **copy that exact prompt into your Vibe Coding tool** and let it inspect the repository and make the Task 1 changes.

After the tool finishes

Do not give it Prompt 2 yet.

Send me its response here, especially:

- files it changed
- explanation of the bug
- changes it made
- any code/error it reports

Then we'll verify whether Task 1 was actually implemented correctly and I'll tell you exactly how to test it.

Your sequence is:

Prompt 1 → AI response → Test Task 1 →  → Prompt 2

So go ahead and run **Prompt 1 now.**

Now work **ONLY** on Task 2 from the README: Add session statistics.

Please first inspect the current code after the Task 1 changes and understand the existing game flow.

Implement the following:

- Track the number of rounds played during the current session.
- Track the number of rounds won during the current session.
- Track the player's best winning streak during the current session.
- Starting a new round must reset only round-specific data.
- Session statistics must **NOT** be reset when starting a new

round.

- Update the statistics at the appropriate points when a round is won or lost.
- Display the session statistics in an appropriate place in the existing game flow.
- Preserve all functionality from Task 1, including the repeated-guess fix.
- Do not implement Task 3 or Task 4 yet.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- Keep the existing project structure and make clean, modular changes rather than rewriting the project.

After making the changes, explain:

1. Which files you changed.
2. How rounds played, rounds won, and best streak are tracked.
3. How you made sure starting a new round does not erase session statistics.
4. How I can manually test all three statistics.

Yes bro — **this is the correct Prompt 2** for Task 2. 👍

Give this prompt to your Vibe Coding tool **only after Task 1 was successfully implemented and tested**.

What you should do now

1. Paste the Prompt 2 you wrote into the Vibe Coding tool.
2. Let it inspect the **current code**, including the Task 1 changes.
3. Let it make the Task 2 changes.
4. **Do not give it Prompt 3 yet.**

After it finishes

Send me the Vibe Coding tool's response/screenshot.

Then we'll test these three things manually:

Test 1 — Rounds Played

</> Plain text

Start game
→ Finish Round 1
→ Finish Round 2
→ Check statistics

Expected:

</> Plain text

Rounds Played: 2

Test 2 — Rounds Won

</> Plain text

Round 1 → WIN
Round 2 → LOSE
Round 3 → WIN

Expected:

</> Plain text

Rounds Played: 3
Rounds Won: 2

Test 3 — Best Winning Streak

For example:

</> Plain text

Round 1 → WIN
Round 2 → WIN
Round 3 → WIN
Round 4 → LOSE

Expected best streak:

</> Plain text

Best Streak: 3

Then start another round and make sure the **round-specific information resets**, but:

</> Plain text

Rounds Played

Rounds Won

Best Streak

do not reset.

That's important because the Task 2 requirement specifically says session statistics must survive when a new round starts.

Your workflow remains:

Task 1  → Task 2 → Test Task 2 → Task 3

So go ahead and give **Prompt 2** to the Vibe Coding tool. Once it responds, paste its response here and I'll check it with you before we move to Task 3.

Now work **ONLY** on Task 3 from the README: Add difficulty selection and scoring.

Please inspect the current implementation and preserve all functionality already implemented in Tasks 1 and 2.

Implement the requirements from Task 3:

- Add difficulty choices to the game.
- Difficulty must affect the number of available lives.
- Difficulty must affect scoring.
- Keep the existing category selection functionality.

- Using a hint must consistently affect the player's score as specified by the README.
- Make sure the scoring behavior is consistent and predictable.
- Difficulty selection should integrate cleanly with the existing game flow.
- Preserve the repeated-guess protection from Task 1.
- Preserve the session statistics from Task 2.
- Do not implement Task 4 yet.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- Do not replace the existing Hangman implementation with an unrelated implementation.
- Make the smallest clean and modular changes necessary.

After making the changes, explain:

1. Which files you changed.
2. What difficulty options were added.
3. How each difficulty changes lives.
4. How each difficulty changes scoring.
5. How hints affect the score.
6. How I can manually test difficulty, category selection, scoring, and hints.

Yes bro 👍 **this is the correct Prompt 3 for Task 3.**

Give this to your Vibe Coding tool **now**, assuming you have already:

- completed Task 1,
- completed and tested Task 2,
- and the current code contains both sets of changes.

Important

Don't let the tool move ahead to Task 4. Your prompt correctly says:

Do not implement Task 4 yet.

After the tool finishes, **don't immediately give it Prompt 4.**

First send me its response. I want to check:

1. Which difficulty levels it actually added.
2. How many lives each level gives.
3. How the scoring formula works.
4. Exactly how the hint changes the score.
5. Whether Task 1 and Task 2 functionality was preserved.

Then we'll manually test:

</> Plain text

```
Difficulty selection
    ↓
Different lives
    ↓
Category selection
    ↓
Correct / wrong guesses
    ↓
Score changes
    ↓
Hint
    ↓
Score changes correctly
    ↓
Session statistics still work
```

One important point: **don't invent the difficulty/lives/scoring values yourself.** The Vibe Coding tool should follow the actual Task 3 requirements in your repository's README. If its response says something that isn't supported by the README, send it to me and we'll check it.

So go ahead and run **Prompt 3**. Once it finishes, paste the response here **before moving to Task 4.**

Now work **ONLY** on Task 4 from the README: Improve input handling.

Please inspect the current implementation and preserve all

functionality from Tasks 1, 2, and 3.

Implement all input-handling requirements from Task 4:

- Handle invalid letter input safely.
- Handle repeated commands safely.
- Handle category selection correctly.
- Handle hint input correctly.
- Invalid input must NOT modify the game state.
- Repeated input should not accidentally change lives, score, guessed letters, streaks, or other game state.
- Feedback/messages should not be duplicated.
- Make sure valid inputs continue to work normally.
- Preserve the repeated-guess fix from Task 1.
- Preserve session statistics from Task 2.
- Preserve difficulty and scoring from Task 3.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- Do not rewrite the project unnecessarily.
- Keep the implementation clean, modular, and consistent with the existing project.

After making the changes, explain:

1. Which files you changed.
2. What invalid inputs are now handled.
3. How repeated commands are handled.
4. How invalid input is prevented from changing game state.
5. How duplicate feedback was prevented.
6. Give me a complete manual testing checklist for Task 4.

Yes bro — **this is the correct Prompt 4 for Task 4.** 👍

Since you've already worked through Tasks 1 → 2 → 3, this prompt correctly tells the Vibe Coding tool to preserve all previous functionality while focusing **only on input handling**.

Give this Prompt 4 to your Vibe Coding tool now

After it finishes, **don't make any more changes immediately**. First send me its response.

Then we'll do the **final testing checklist** for the complete project.

Task 4 testing

We'll test these cases:

Test	What to check
Invalid letter	Game doesn't crash or change state
Number input	No life/score/state change
Special character	No state change
Empty input	Safely rejected
Multiple characters	Safely rejected
Repeated wrong letter	No second life deduction
Repeated correct letter	Doesn't count again
Category input	Correct category is selected
Invalid category	Doesn't alter game state
Hint	Works and applies the Task 3 scoring rule
Repeated hint	Doesn't incorrectly alter state
Valid guess	Works normally
Messages	No duplicate feedback
Statistics	Tasks 1–3 statistics remain correct
Difficulty	Lives/scoring remain correct



One important thing

Don't record the after video yet.

First we need to make sure all four tasks work together:

</> Plain text

Task 1 → Repeated guesses 
Task 2 → Session statistics 
Task 3 → Difficulty + scoring 
Task 4 → Input handling 

↓

Final testing

↓

AFTER video

↓

Git commit

↓

Git push

Your lab requires the **after video to be recorded after all changes are completed**, and the final submission also needs the updated code and exported chat history.



Lab_4_VibeCoding_Student_handou...

So go ahead and run **Prompt 4**. Then paste the Vibe Coding tool's response here, and I'll take you through the **final testing → after video → GitHub push → Lab-4 submission** step by step.

I am working on this existing Python Hangman project. Please inspect the existing code and README before making changes.

For this prompt, work ONLY on Task 1 from the README: Fix repeated guesses.

Requirements:

- A repeated wrong letter must NOT reduce the player's lives again.
- A repeated correct letter must NOT be counted as another guess.
- Every distinct guessed letter should affect the current round only once.
- The existing game flow and functionality should be preserved.
- Do not implement Task 2, Task 3, or Task 4 yet.
- Do not rewrite the project unnecessarily.
- Make the smallest clean and modular changes needed.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- After making the changes, explain:
 1. Which files you changed.
 2. What was causing the repeated-guess problem.
 3. How your fix prevents repeated guesses from affecting the round twice.
 4. How I can test the fix manually.

Worked for 11s

I inspected the files currently available to me. I can see the Hangman assignment context and the previous chat-history PDF, which confirms the project contains `main.py` , `game.py` , `stats.py` , `words.py` , `README.md` , and `requirements.txt` .  ChatGPT_Hangman_Chat_History

However, **the actual Python source files and README are not currently available as files for me to edit**. I don't want to guess at the implementation and accidentally modify Tasks 2–4 or

rewrite something unnecessarily.

Please **upload the current Hangman project folder/ZIP** (or at least these files):

- README.md
- main.py
- game.py
- stats.py
- words.py
- requirements.txt if present

Once you upload them, I'll:

1. Read the README and identify **exactly what Task 1 requires**.
2. Inspect the existing implementation before changing anything.
3. Modify **only the files actually needed for Task 1**.
4. Keep the existing game flow and Tasks 2–4 untouched.
5. Test the repeated-guess behavior.
6. Give you a clear summary of:
 - files changed,
 - root cause,
 - exact fix,
 - manual testing steps.

I won't add any CSV/JSON/database persistence or unnecessary refactoring.

 https://github.com/nitamalkhedkar/16_hangman.git

Worked for 8s

I inspected your actual GitHub repository and the README. [16_hangman GitHub repository ↗](#)

What I found

The README's **Task 1** specifically says that each distinct letter should affect the round exactly once.  [GitHub](#)

The problem is in `game.py`, inside `HangmanGame.guess()`.

Currently, the code checks:

</> Python

```
if letter in self.guessed:  
    return "Already guessed."
```

But `self.guessed` contains only **correct** letters.

For a wrong letter, the code puts it into:

</> Python

```
self.wrong.add(letter)  
self.lives -= 1
```

There is **no check for** `self.wrong` **before reducing lives**. So:

</> Plain text

Guess: z → Wrong → lives 5

Guess: z → Wrong → lives 4 

Guess: z → Wrong → lives 3 

Minimal Task 1 fix

Only `game.py` needs to change.

The relevant logic should become:

</> Python

```
def guess(self, letter):  
    if len(letter) != 1 or not letter.isalpha():  
        return "Enter one letter."  
  
    if letter in self.guessed or letter in self.wrong:  
        return "Already guessed."  
  
    if letter in self.secret:  
        self.guessed.add(letter)  
        return "Correct."
```

```
self.wrong.add(letter)
self.lives -= 1
return "Wrong."
```

This is a **very small change**: the existing `wrong` set is already there, so there is no need to introduce another data structure or rewrite the game.

The existing correct-guess protection remains intact, while the new `letter in self.wrong` check prevents a previously incorrect letter from consuming another life.  [GitHub](#)

Important: I have not changed the GitHub repository yet

I can inspect the public repository, but I don't currently have permission to push/edit your GitHub repository from this chat. I suggested the GitHub connection so that, once you connect it, I can make the change directly rather than asking you to manually copy it.

I have **not touched Tasks 2, 3, or 4**, and I would not modify `main.py` , `stats.py` , `words.py` , or `README.md` for this Task 1 fix.

How to manually test Task 1

Run:

</> Bash

```
python main.py
```

Choose a category and start a round.

Test 1 — repeated wrong letter

Suppose the word doesn't contain `z` :

</> Plain text

```
Guess z
```

You should see:

</> Plain text

Wrong.

Lives: 5

Then enter `z` again.

It should **not** reduce lives:

</> Plain text

Already guessed.

Lives: 5

Test 2 — repeated correct letter

If the word contains `e` :

</> Plain text

Guess `e`

You should get:

</> Plain text

Correct.

Enter `e` again.

It should say:

</> Plain text

Already guessed.

and the word/lives should remain unchanged.

Test 3 — different letters

For example:

⟨/⟩ Plain text

```
z → lives 5
x → lives 4
z → still lives 4
x → still lives 4
```

So each distinct letter affects the current round **only once**.

The existing round state is already reset in `start_round()` using `self.guessed.clear()` and `self.wrong.clear()`, so the protection naturally applies to the **current round only**.



Now work **ONLY** on Task 2 from the README: Add session statistics.

Please first inspect the current code after the Task 1 changes and understand the existing game flow.

Implement the following:

- Track the number of rounds played during the current session.
- Track the number of rounds won during the current session.
- Track the player's best winning streak during the current session.
- Starting a new round must reset only round-specific data.
- Session statistics must **NOT** be reset when starting a new round.
- Update the statistics at the appropriate points when a round is won or lost.
- Display the session statistics in an appropriate place in the existing game flow.

- Preserve all functionality from Task 1, including the repeated-guess fix.
- Do not implement Task 3 or Task 4 yet.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- Keep the existing project structure and make clean, modular changes rather than rewriting the project.

After making the changes, explain:

1. Which files you changed.
2. How rounds played, rounds won, and best streak are tracked.
3. How you made sure starting a new round does not erase session statistics.
4. How I can manually test all three statistics.

Worked for 14s

I inspected the existing Task 1 structure and the session flow. For Task 2, the cleanest approach is to keep **session-level statistics inside the existing HangmanGame object** and keep `start_round()` responsible only for round-specific state.

I can give you the exact Task 2 patch, but I **cannot push the change directly to your GitHub repository from this chat right now** because the GitHub connection needed for write access isn't active.

Task 2 changes

File changed: `game.py` only

Your existing `stats.py` does not need to be rewritten. The statistics belong to the active `HangmanGame` session.

1. Add session statistics in `__init__()`

Keep your existing Task 1 fields and add:

Python

```
self.rounds_played = 0
self.rounds_won = 0
self.best_streak = 0
```

So the beginning of `__init__()` becomes:

</> Python

```
def __init__(self):
    self.score = 0
    self.streak = 0

    # Session statistics
    self.rounds_played = 0
    self.rounds_won = 0
    self.best_streak = 0

    self.category = "technology"
    self.secret = ""
    self.guessed = set()
    self.wrong = set()
    self.lives = 6
    self.hint_used = False
```

2. Do NOT reset these in `start_round()`

Keep:

</> Python

```
def start_round(self):
    self.secret = random.choice(WORDS[self.category])
    self.guessed.clear()
    self.wrong.clear()
    self.lives = 6
    self.hint_used = False
```

Notice that we **do not** do:

</> Python

```
self.rounds_played = 0
```

```
self.rounds_won = 0
self.best_streak = 0
```

That is important because these are **session statistics**, not round statistics.

3. Add a method to record the result of a round

Add this to HangmanGame :

</> Python

```
def record_round_result(self, won):
    self.rounds_played += 1

    if won:
        self.rounds_won += 1
        self.streak += 1
        self.best_streak = max(self.best_streak, self.streak)
    else:
        self.streak = 0
```

This gives us:

- Every completed round → rounds_played + 1
- Winning round → rounds_won + 1
- Consecutive win → streak + 1
- Highest streak ever reached → best_streak
- Losing → current streak = 0
- best_streak is **not** reset after a loss

For example:

</> Plain text

```
WIN → streak 1, best 1
WIN → streak 2, best 2
WIN → streak 3, best 3
LOSS → streak 0, best 3
WIN → streak 1, best 3
```

4. Preserve the Task 1 repeated-guess fix

Your `guess()` method should still contain:

</> Python

```
if letter in self.guessed or letter in self.wrong:  
    return "Already guessed."
```

So Task 2 does **not** interfere with Task 1.

5. Update statistics when the round actually ends

In `play_round()`, where the existing code determines whether the player has won or lost, call:

</> Python

```
self.record_round_result(True)
```

when the player wins, and:

</> Python

```
self.record_round_result(False)
```

when the player loses.

The important point is that this happens **once per completed round**, not once per guess.

So the flow should conceptually be:

</> Plain text

```
start_round()  
  ↓  
player guesses  
  ↓  
guess()  
  ↓  
check won/lost  
  ↓  
round finished
```

```
↓  
record_round_result(True/False)  
↓  
display session statistics
```

6. Display the statistics

Add a small helper method:

</> Python

```
def display_session_stats(self):  
    print("\nSession Statistics")  
    print(f"Rounds played: {self.rounds_played}")  
    print(f"Rounds won: {self.rounds_won}")  
    print(f"Best winning streak: {self.best_streak}")
```

Then call it after a round finishes.

For example:

</> Plain text

You won! The word was: python

```
Session Statistics  
Rounds played: 1  
Rounds won: 1  
Best winning streak: 1
```

After another win:

</> Plain text

```
Session Statistics  
Rounds played: 2  
Rounds won: 2  
Best winning streak: 2
```

After losing the next round:

</> Plain text

```
Session Statistics
Rounds played: 3
Rounds won: 2
Best winning streak: 2
```

Why this design is correct

There are now two clearly separated types of state:

Round-specific state

</> Python

```
self.secret
self.guessed
self.wrong
self.lives
self.hint_used
```

These are reset by:

</> Python

```
start_round()
```

Session-specific state

</> Python

```
self.rounds_played
self.rounds_won
self.streak
self.best_streak
```

These live for the lifetime of the `HangmanGame` object.

Therefore:

</> Plain text

New round

↓

reset secret/guesses/lives/hint

↓

KEEP session statistics

No CSV, JSON, SQLite, database, or other persistent storage is introduced.

Manual testing

Test 1 — Rounds played

Complete three rounds:

</> Plain text

Round 1 → WIN

Round 2 → LOSS

Round 3 → WIN

Expected:

</> Plain text

Rounds played: 3

Rounds won: 2

Test 2 — Rounds won

Play:

</> Plain text

WIN

LOSS

WIN

LOSS

Expected:

</> Plain text

Rounds played: 4

Rounds won: 2

A loss must never increase `rounds_won` .

Test 3 — Best winning streak

Play:

</> Plain text

WIN

WIN

WIN

LOSS

WIN

Expected:

</> Plain text

Rounds played: 5

Rounds won: 4

Best winning streak: 3

The loss resets the **current** streak to zero, but it must **not** erase the best streak of 3.

Test 4 — New round must preserve statistics

After getting:

</> Plain text

Rounds played: 3

Rounds won: 2

Best winning streak: 2

start another round.

The new round should begin with fresh:

⟨/⟩ Plain text

```
word
guessed letters
wrong letters
lives
hint state
```

but the statistics must remain:

⟨/⟩ Plain text

```
Rounds played: 3
Rounds won: 2
Best winning streak: 2
```

Final Task 2 scope

File	Change
game.py	Add session counters, result tracking, statistics display, preserve Task 1
main.py	No change
words.py	No change
stats.py	No change
requirements.txt	No change
Persistent storage	None
Task 3	Not implemented
Task 4	Not implemented

I wasn't able to write/push these changes to GitHub because the repository's write connection isn't available in this chat; the clone attempt also couldn't reach GitHub. If you connect GitHub, I can apply the patch directly and verify the resulting files.

Which next step do you want: GitHub connection or local patch ?

Now work ONLY on Task 3 from the README: Add difficulty selection and scoring.

Please inspect the current implementation and preserve all functionality already implemented in Tasks 1 and 2.

Implement the requirements from Task 3:

- Add difficulty choices to the game.
- Difficulty must affect the number of available lives.
- Difficulty must affect scoring.
- Keep the existing category selection functionality.

- Using a hint must consistently affect the player's score as specified by the README.
- Make sure the scoring behavior is consistent and predictable.
- Difficulty selection should integrate cleanly with the existing game flow.
- Preserve the repeated-guess protection from Task 1.
- Preserve the session statistics from Task 2.
- Do not implement Task 4 yet.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- Do not replace the existing Hangman implementation with an unrelated implementation.
- Make the smallest clean and modular changes necessary.

After making the changes, explain:

1. Which files you changed.
2. What difficulty options were added.
3. How each difficulty changes lives.
4. How each difficulty changes scoring.
5. How hints affect the score.
6. How I can manually test difficulty, category selection, scoring, and hints.

I can work on Task 3, but I need to be careful about one important point: **the exact difficulty/lives/scoring rules must come from your README**, and the current GitHub repository isn't accessible through the available write-capable connection in this chat right now. I don't want to invent scoring values and accidentally implement something different from the assignment.

From the Task 1/2 implementation already inspected, I know the existing structure includes:

- `game.py` — main HangmanGame logic
- `main.py` — starts the game
- `words.py` — categories/words/hints
- `stats.py` — session statistics

- Task 1 repeated-guess protection:

```
</> Python

if letter in self.guessed or letter in self.wrong:
    return "Already guessed."
```

- Task 2 session fields such as:

```
</> Python

self.rounds_played
self.rounds_won
self.best_streak
```

What I need before modifying Task 3

Please **paste the Task 3 section of your README here** (just that section is enough), especially the exact requirements for:

- difficulty names
- lives for each difficulty
- points/scoring for each difficulty
- hint score penalty/cost

For example, I don't want to assume something like:

```
</> Plain text

Easy    = 6 lives
Medium  = 5 lives
Hard    = 3 lives
```

unless that's actually what your README specifies.

Once you paste **Task 3 from the README**, I can give you the exact minimal changes to `game.py` /other required files while preserving **Tasks 1 + 2** and leaving **Task 4 completely untouched**.

Now work **ONLY** on Task 4 from the README: Improve input

...

handling.

Please inspect the current implementation and preserve all functionality from Tasks 1, 2, and 3.

Implement all input-handling requirements from Task 4:

- Handle invalid letter input safely.
- Handle repeated commands safely.
- Handle category selection correctly.
- Handle hint input correctly.
- Invalid input must NOT modify the game state.
- Repeated input should not accidentally change lives, score, guessed letters, streaks, or other game state.
- Feedback/messages should not be duplicated.
- Make sure valid inputs continue to work normally.
- Preserve the repeated-guess fix from Task 1.
- Preserve session statistics from Task 2.
- Preserve difficulty and scoring from Task 3.
- Do not add persistent storage such as CSV, JSON, SQLite, or any database.
- Do not rewrite the project unnecessarily.
- Keep the implementation clean, modular, and consistent with the existing project.

After making the changes, explain:

1. Which files you changed.
2. What invalid inputs are now handled.
3. How repeated commands are handled.
4. How invalid input is prevented from changing game state.
5. How duplicate feedback was prevented.
6. Give me a complete manual testing checklist for Task 4.

Yes. For **Task 4 only**, I would preserve Tasks 1–3 and make the input layer responsible for validation, while keeping the existing game logic intact.

However, I need the **actual current Task 3 implementation** before I can safely give you a patch. In the previous turn, the Task 3 changes were not actually applied to the repository, so I cannot honestly claim to be inspecting a codebase that contains Tasks 1–3.

Please paste or upload your current:

- `game.py`
- `main.py`
- `stats.py` (if you changed it for Task 2)

or connect the GitHub repository for editing.

Once I have the current code, I'll make **only Task 4 changes** and specifically verify:

- invalid letters don't consume lives or alter score
- empty input is harmless
- numbers/symbols/multiple characters are rejected safely
- repeated correct guesses don't alter state
- repeated wrong guesses don't alter state
- `/hint` behaves safely when repeated
- `/quit` behaves safely when repeated
- invalid category selections don't change the category
- valid category selection still works
- invalid commands don't accidentally trigger hints/guesses
- feedback is printed exactly once per input
- Task 1 repeated-guess protection remains intact
- Task 2 statistics remain intact
- Task 3 difficulty/lives/scoring remain intact
- **no persistent storage**
- **no unnecessary rewrite**

I also won't implement or modify anything outside **Task 4**.

[next](#)

Sure. Send me the **current** `game.py` and `main.py` (and `stats.py` if you modified it), or connect the GitHub repo.

Then I'll proceed directly with **Task 4 only** and give you:

1. Exact files changed
2. Complete input-handling fixes
3. Protection against state changes from invalid/repeated input
4. Duplicate-feedback fix
5. Full Task 4 manual testing checklist

I'll preserve Tasks **1, 2, and 3** and won't touch persistent storage or rewrite the project.